API Testimine Praktika Projekt – REST & SOAP testimine

Eesmärk: Projektis testisin REST ja SOAP API-sid eesmärgina hinnata teenuste töökindlust, vastavust dokumentatsioonile ning süsteemi käitumist erinevate sisendite korral.

Fookuses olid nii positiivsed kui negatiivsed teststsenaariumid, veahandluse kontroll ning API funktsionaalsuse analüüs.

Kasutatud tööriistad:

- Postman - REST API päringute saatmine ja vastuste valideerimine
- SoapUI - SOAP teenuste testimine
- Swagger Petstore API - REST API testkeskkond
- Calculator SOAP API - SOAP teenuse funktsionaalsuse testimine
- JSON vastuste analüüs
- HTTP staatusekoodide kontroll

Mida testisin:

REST API testimine

- GET päringud olemasolevate ja mitteolemasolevate ressurssidega
- POST päring failide üleslaadimiseks
- DELETE operatsioonide korrektne toimimine
- Andmete filtreerimine päringuparameetrite abil
- HTTP vastuskoodide ja API vastuste kontroll
- API käitumine vigaste või puuduvate andmete korral

SOAP API testimine

- Matemaatiliste operatsioonide korrektsus (Add, Divide, Multiply, Subtract)
- Positiivsed ja negatiivsed sisendid
- Kümnendmurdude tugi
- Suurte arvude töötlemine
- Tulemuste täpsus ja ümardamisloogika

Mida õppisin?

- REST ja SOAP API testimise põhimõtted praktikas
- Kuidas koostada positiivseid ja negatiivseid testjuhtumeid
- HTTP staatusekoodide ja API vastuste analüüsimine
- API dokumentatsiooni kasutamine testimise alusena
- Süsteemi piirangute ja loogikavigade tuvastamine
- Erinevused REST ja SOAP arhitektuuride vahel
- Testimise käigus leitud probleemide dokumenteerimine ja järelduste tegemine

1. REST API testimine – Swagger Pet Store (Postman)

URL: https://petstore.swagger.io/

Eesmärk: Kontrollida REST API lõpp-punktide korrektset toimimist vastavalt Swaggeri dokumentatsioonile. Iga lõpp-punkti tuleb testida nii positiivsete kui negatiivsete tulemustega.

Testjuhtum 1 A – GET /pet/{petId} (olemasolev ID)

URL: https://petstore.swagger.io/v2/pet/10

Sisend: petId = 10**,**

Request body: puudub

Oodatav tulemus: HTTP 200 OK

JSON objekt, mis sisaldab vähemalt järgmisi välju: Id, category, name, photoUrls, tags, status

Tegelik tulemus: HTTP 200 OK

Järeldus: API käitub ootuspäraselt olemasoleva ID puhul.

https://petstore.swagger.io/v2/pet/10

Save | Share

GET https://petstore.swagger.io/v2/pet/10 Send

Docs | Params | Authorization | Headers 6 | Body | Scripts | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | Cookies | Headers 7 | Test Results

200 OK • 128 ms • 426 B

{} JSON | Preview | Visualize


```
{
    "id": 10,
    "category": {
        "id": 10,
        "name": "string"
    },
    "name": "doggie",
    "photoUrls": [
        "string"
    ],
    "tags": [
        {
            "id": 10,
            "name": "string"
        }
    ],
    "status": "string"
}
```

Testjuhtum 1 B – GET /pet/{petId} (mitteolemasolev ID)

URL: https://petstore.swagger.io/v2/pet/0

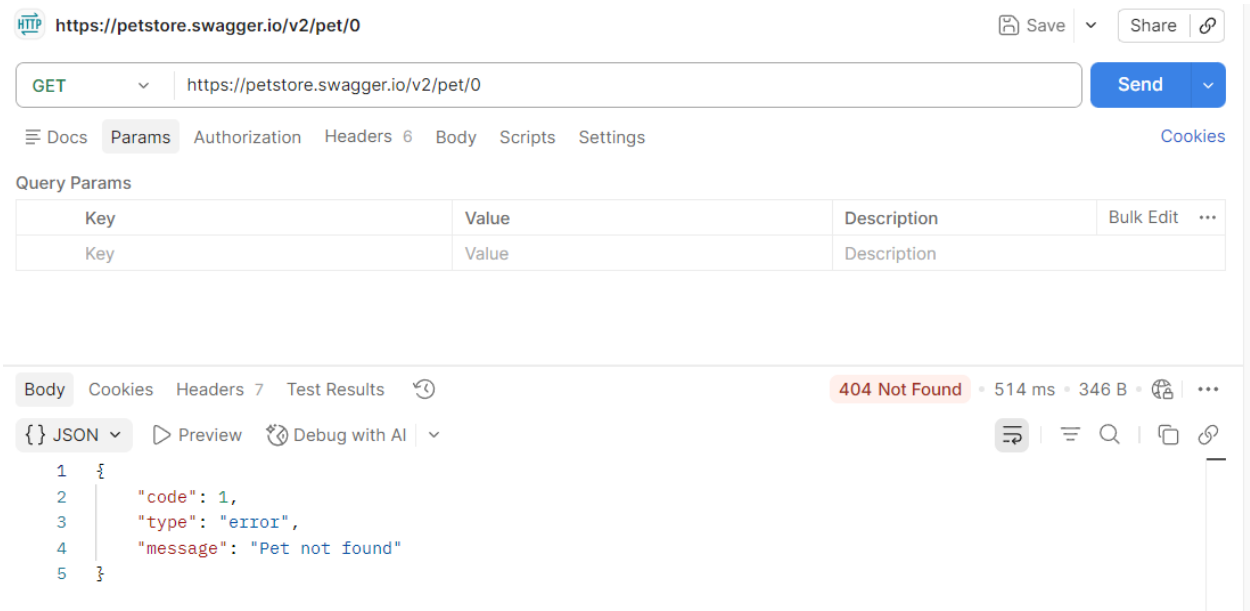
Sisend: petId = 0,

Request body: puudub

Oodatav tulemus: HTTP 404 ja mingi veateade

Tegelik tulemus: HTTP 404 ja JSON vastus veateatega „Pet not found“

Järeldus: API käitub ootuspäraselt mitteolemasoleva ressursi puhul.



Testjuhtum 2 – POST /pet (uue looma lisamine petId-ga)

URL: https://petstore.swagger.io/v2/pet

Sisend:

Request body:

```
1  {
2    "id": 1,
3    "category": {
4      "id": 1,
5      "name": "string"
6    },
7    "name": "doggie",
8    "photoUrls": [
9      "string"
10   ],
11   "tags": [
12     {
13       "id": 1,
14       "name": "string"
15     }
16   ],
17   "status": "available"
18 }
```

Oodatav tulemus: HTTP 200 OK ja mingi kinnitus, et uus loom sai lisatud.

Tegelik tulemus: HTTP 200 OK, Server tagastas Pet objekti, mis saadeti request body's, JSON struktuur vastab Pet mudelile, Server tagastab:

- `id: 1`
- `name: "doggie"`
- `status: "available"`

ülejäänud väljad vastavalt sisendile

Järeldus: Päring õnnestus. Uus loom salvestati edukalt ning API tagastas korrektse objekti.

Test läbitud.

https://petstore.swagger.io/v2/pet

Save | Share

POST https://petstore.swagger.io/v2/pet Send

Docs | Params | Authorization | Headers 8 | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
{
  "id": 1,
  "category": {
    "id": 1,
    "name": "string"
  },
  "name": "doggie",
  "photoUrls": [
```

Body | Cookies | Headers 7 | Test Results

200 OK · 506 ms · 426 B

{} JSON | Preview | Visualize

```
{
    "id": 1,
    "category": {
        "id": 1,
        "name": "string"
    },
    "name": "doggie",
    "photoUrls": [
        "string"
    ],
    "tags": [
        {
            "id": 1,
            "name": "string"
        }
    ],
    "status": "available"
}
```

Testjuhtum 3 – POST /store/order (Tellimuse loomine)

URL: https://petstore.swagger.io/v2/store/order

Sisend:

Request Body:

```
1  {
2    "id": 12345,
3    "petId": 1,
4    "quantity": 1,
5    "shipDate": "2024-01-01T00:00:00.000Z",
6    "status": "placed",
7    "complete": true
8  }
```

Oodatav tulemus: HTTP 200 OK, mingi tellimus on loodud

Tegelik tulemus: HTTP 200 OK, Server tagastas identse tellimuse, ainult shipDate oli teisendatud: 2024-01-01T00:00:00.000+0000

Järeldus: Tellimus loodi edukalt. API tagastas korrektsed andmed ning serveri kuupäevavormingu. Test läbitud.

https://petstore.swagger.io/v2/store/order

Save | Share

POST https://petstore.swagger.io/v2/store/order Send

Docs | Params | Authorization | Headers 9 | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
{
  "id": 12345,
  "petId": 1,
  "quantity": 1,
  "shipDate": "2024-01-01T00:00:00.000Z",
  "status": "placed",
  "complete": true
}
```

Body | Cookies | Headers 7 | Test Results

200 OK • 544 ms • 399 B

{} JSON | Preview | Visualize

```
{
  "id": 12345,
  "petId": 1,
  "quantity": 1,
  "shipDate": "2024-01-01T00:00:00.000+0000",
  "status": "placed",
  "complete": true
}
```

Testjuhtum 4 A – DELETE /store/order /{orderId} (eksisteeriv order)

URL: https://petstore.swagger.io/v2/store/order/12345

Request body: puudub

Oodatav tulemus: HTTP 200 OK. Mingi kinnitus, et eksisteeriv order kustutati ära

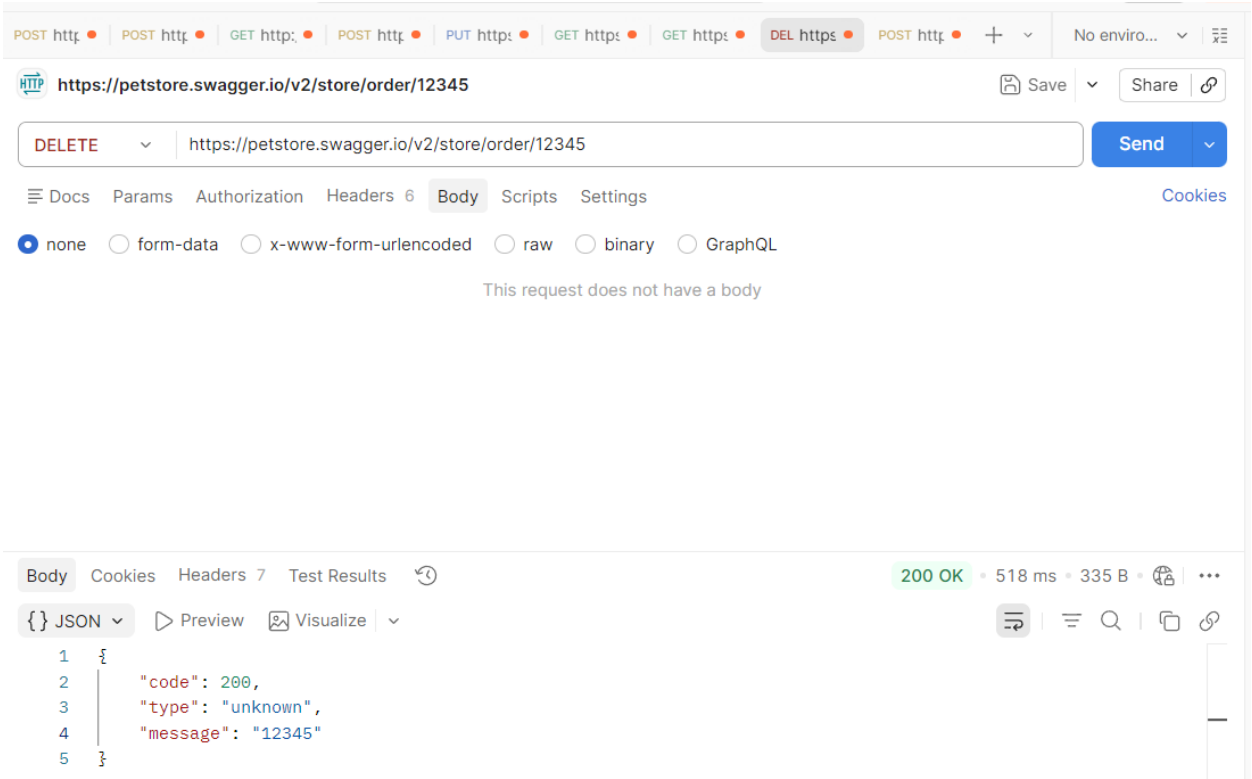
Tegelik tulemus: HTTP 200 OK

Vastus sisaldab (JSON):

```
{
  "code": 200,
  "type": "unknown",
  "message": "12345"
}
```

Nüüd me võime eeldada, et order sai ära kustutatud ja kui me kutsume välja GET /store/order/{orderId}, siis me peame saama vastuseks HTTP 404 Not Found.
Järeldus: tellimus kustutati edukalt ning, sest GET tagastas HTTP 404 Not Found.

Test läbitud.



Testjuhtum 4 B – DELETE /store/order /{orderId} (mitte-eksisteeriv order)

URL: https://petstore.swagger.io/v2/store/order/12345

Sisend: orderId: 12345,

Request body: puudub

Oodatav tulemus: HTTP 404 Not Found ja mingi veateade

Tegelik tulemus: HTTP 404 Not Found

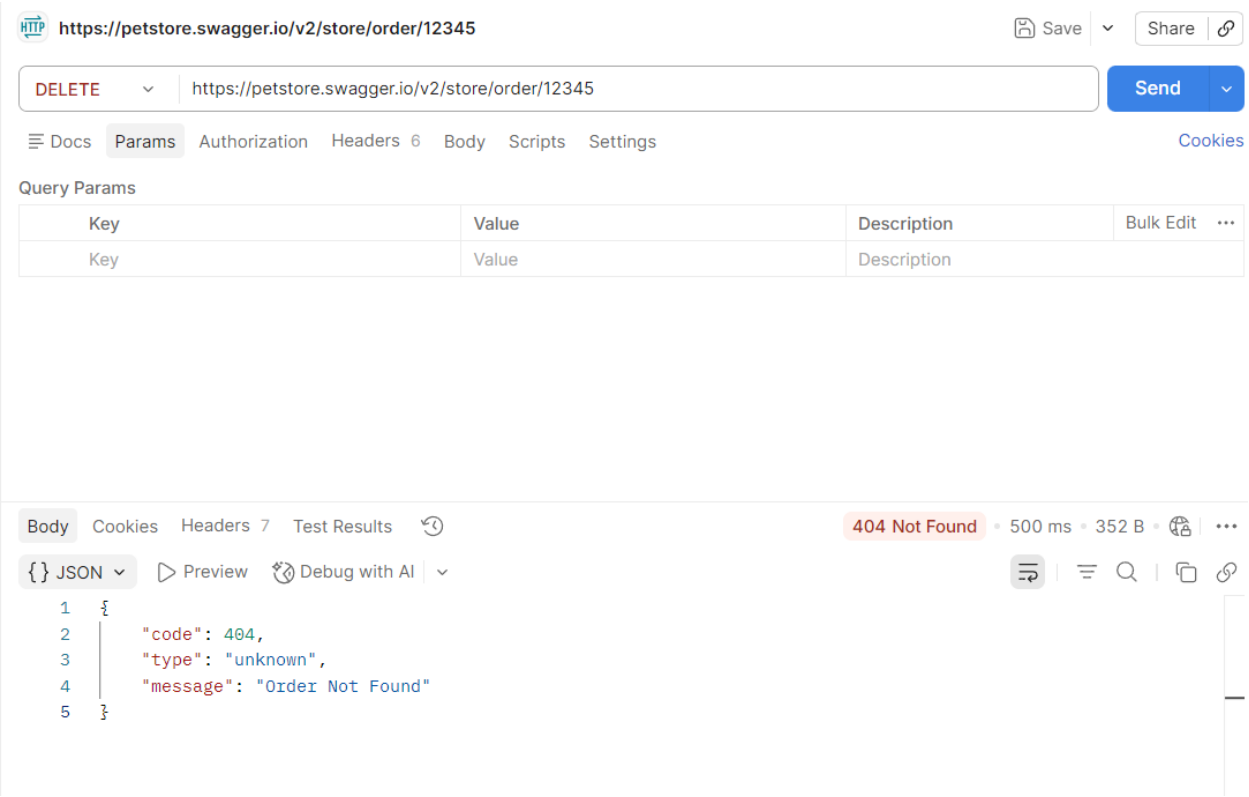
Vastuse JSON sisaldab:

- `code: 404`
- `message: "Order Not Found"`

Järeldus: Tellimust ID-ga 12345 ei eksisteerinud, seega API tagastas korrektse veateate.

See kinnitab, et API käitub õigesti, kui kustutada üritatakse mitte-eksisteerivat tellimust.

Test läbitud — API käitub ootuspäraselt.



Testjuhtum 5 – PUT/user/{username} (kasutaja uuendamine)

URL: https://petstore.swagger.io/v2/user/Bella

Sisend:

Path-parameeter: username: Bella

Request body:

```
1  {
2    "id": 1,
3    "username": "Bella",
4    "firstName": "Bella",
5    "lastName": "Anderson",
6    "email": "bella@example.com",
7    "password": "password123",
8    "phone": "555-1234",
9    "userStatus": 1
10 }
```

Oodatav tulemus: HTTP 200 OK, kasutaja andmed edukalt uuendatud

Tegelik tulemus: HTTP 200 OK

Vastuse JSON sisaldab:

```
code: 200,

type: "unknown",

message: "1"
```

See kinnitab, et Kasutaja andmed uuendatakse serveris.

Järeldus: Kasutaja Bella andmed uuendati edukalt.

PUT päring töötab korrektselt. Test läbitud.

https://petstore.swagger.io/v2/user/Bella

Save | Share

PUT https://petstore.swagger.io/v2/user/Bella Send

Docs | Params | Authorization | Headers 8 | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify


```
{
  "id": 1,
  "username": "Bella",
  "firstName": "Bella",
  "lastName": "Anderson",
  "email": "bella@example.com",
  "password": "password123",
  "phone": "555-1234",
  "userStatus": 1
}
```


Body | Cookies | Headers 7 | Test Results

200 OK · 499 ms · 331 B

{} JSON | Preview | Visualize


```
{
  "code": 200,
  "type": "unknown",
  "message": "1"
}
```

Testjuhtum 6 – GET/user/{username} (kasutaja andmete pärimine)

URL: https://petstore.swagger.io/v2/user/Bella

Sisend:

Path-parameeter: username: Bella

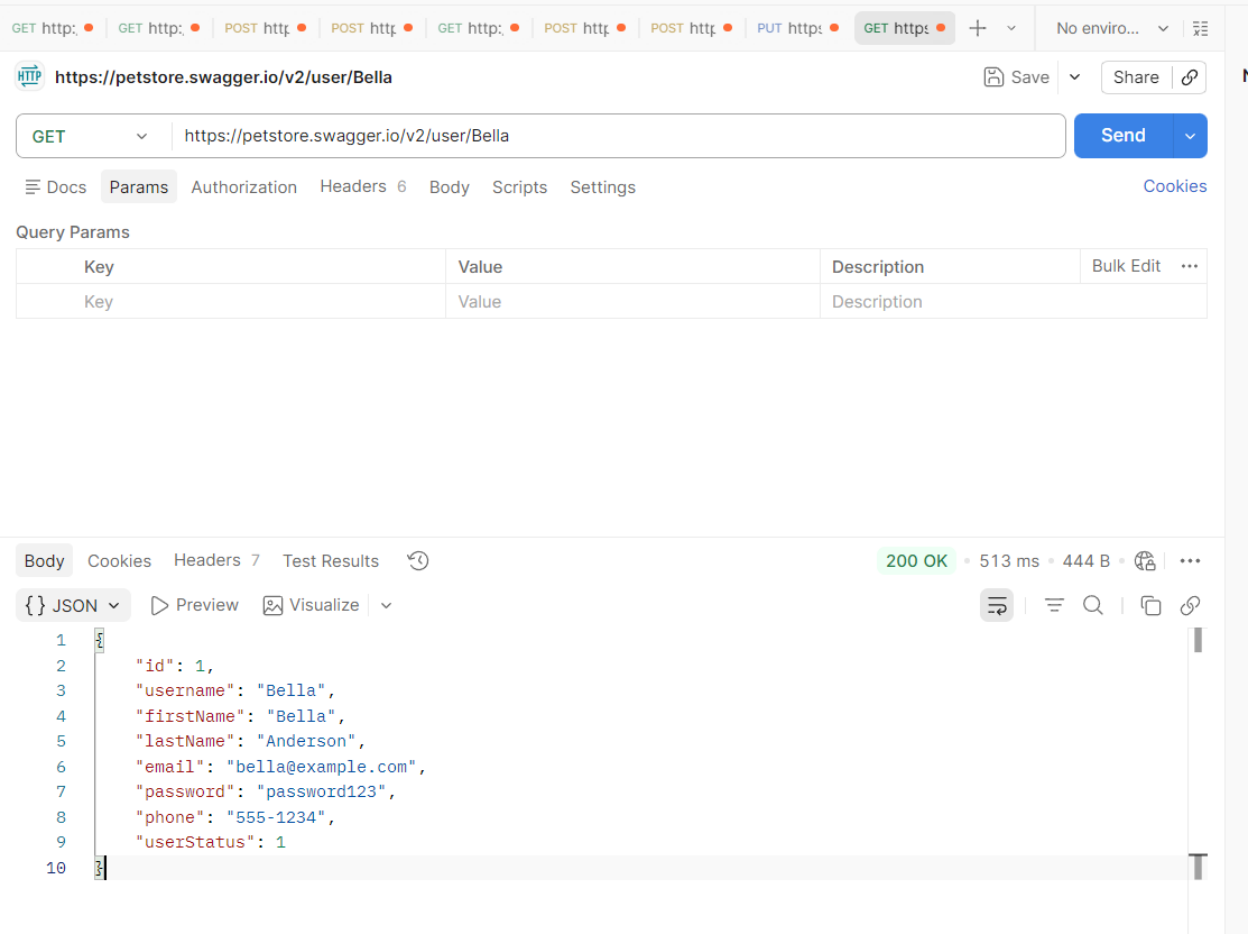
Request body: puudub

Oodatav tulemus: HTTP 200 OK , kasutaja andmed

Tegelik tulemus: HTTP 200 OK , kasutaja andmed

Järeldus: GET päring tagastas uuendatud kasutaja andmed edukalt.

Kasutaja Bella on süsteemis olemas ja andmed vastavad PUT päringule. Test läbitud.



SOAP API testimine – Calculator API (SOAPUI)

WSDL: http://www.dneonline.com/calculator.asmx?wsdl

Eesmärk: SOAP API puhul analüüsisin matemaatiliste operatsioonide täpsust ja süsteemi piiranguid erinevate andmetüüpidega.

Test 1A – Add (2 + 2)

Oodatav tulemus: 4

Tegelik tulemus: 4

Järeldus: Test Läbitud

Test 1B – Add (-2 + 2)

Oodatav tulemus: 0

Tegelik tulemus: 0

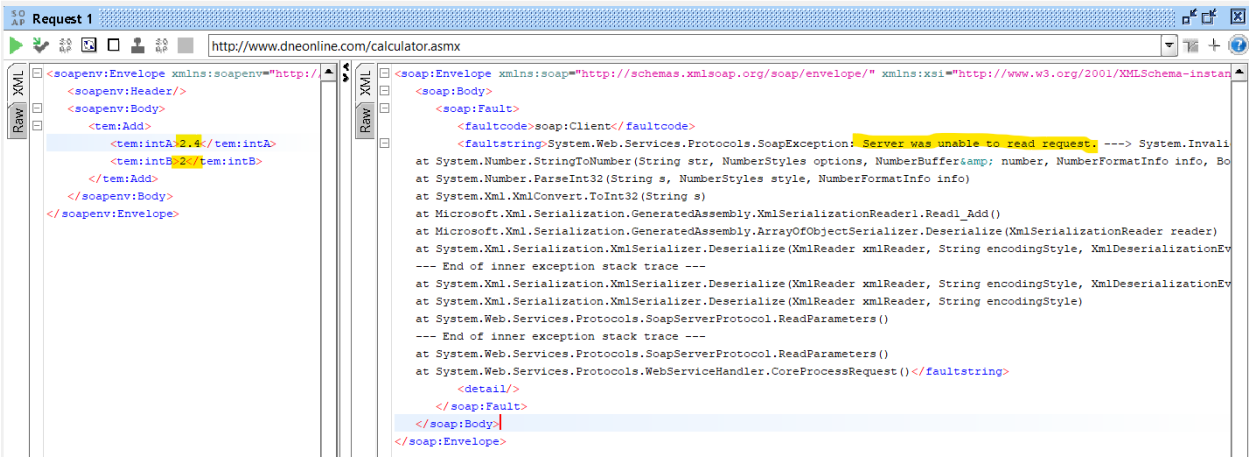
Järeldus: Test Läbitud

Test 2A – Add (2.4 + 2)

Oodatav tulemus: 4.4

Tegelik tulemus: Ebaõnnestus

Järeldus: Süsteem ei toeta positiivseid kümnendmurde.

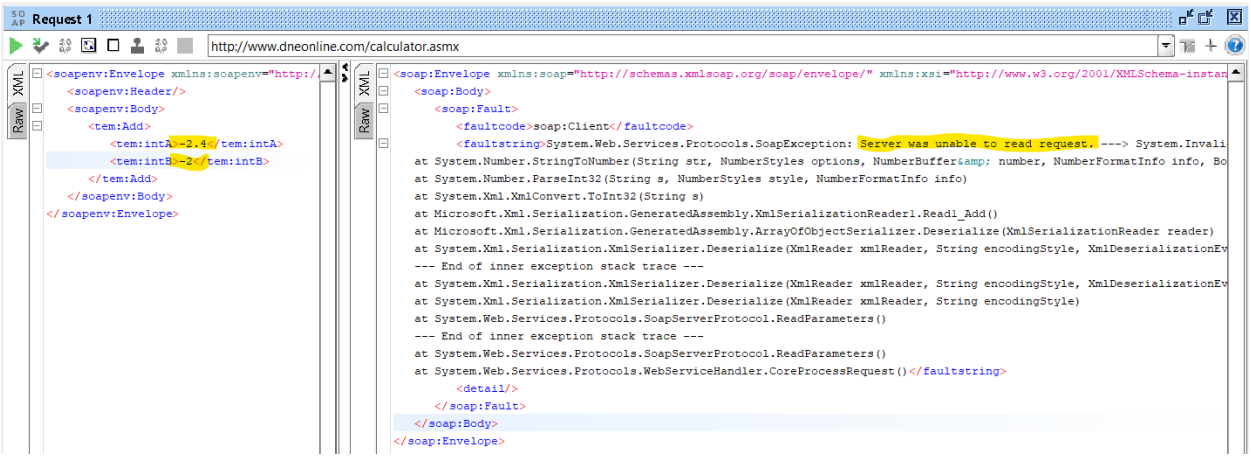


Test 2B – Add (-2.4 + (-2))

Oodatav tulemus: –4.4

Tegelik tulemus: Ebaõnnestus

Järeldus: Süsteem ei toeta negatiivseid kümnendmurde.

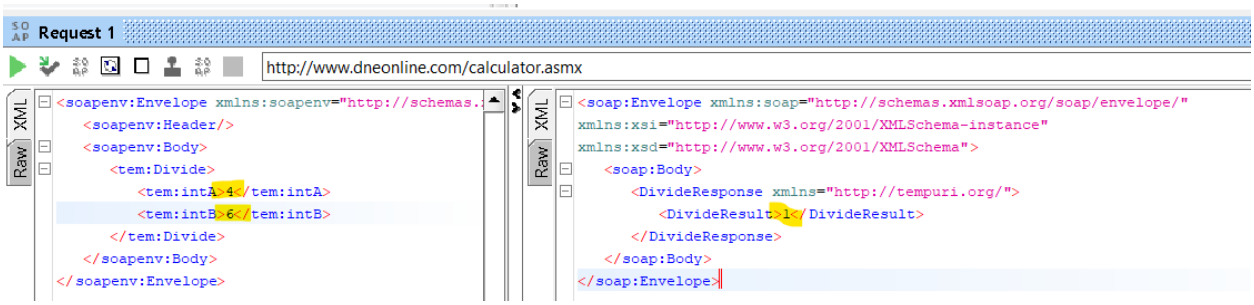


Test 3 – Divide (4 / 6)

Oodatav tulemus: 0.66…

Tegelik tulemus: 1

Järeldus: Tulemus ümardatakse täisarvuks - võimalik loogikaviga.



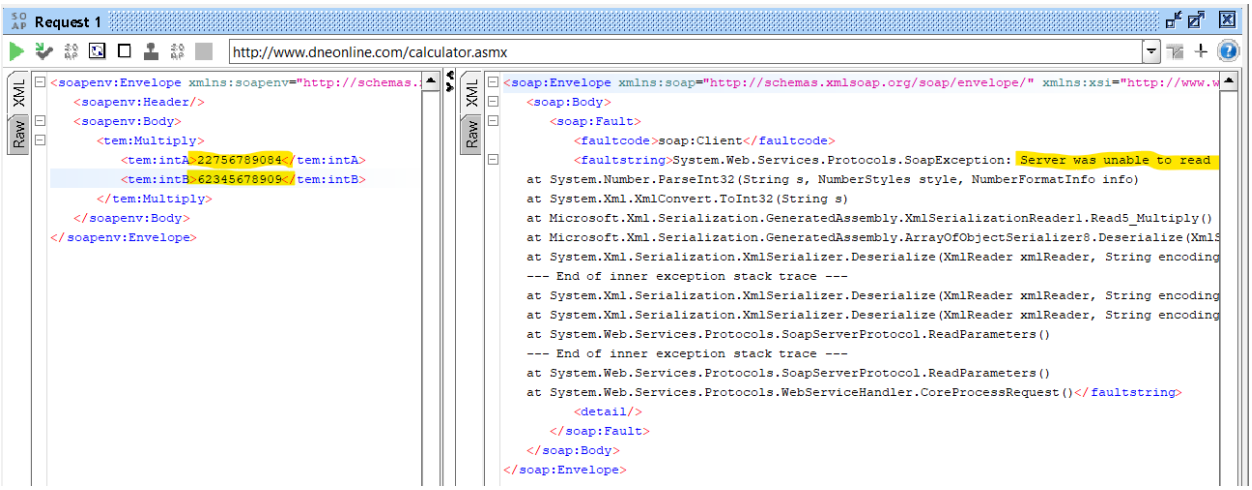
Test 4 – Multiply (suured täisarvud)

Sisend: 22756789084 × 62345678909

Oodatav tulemus: 1.41878747E+21

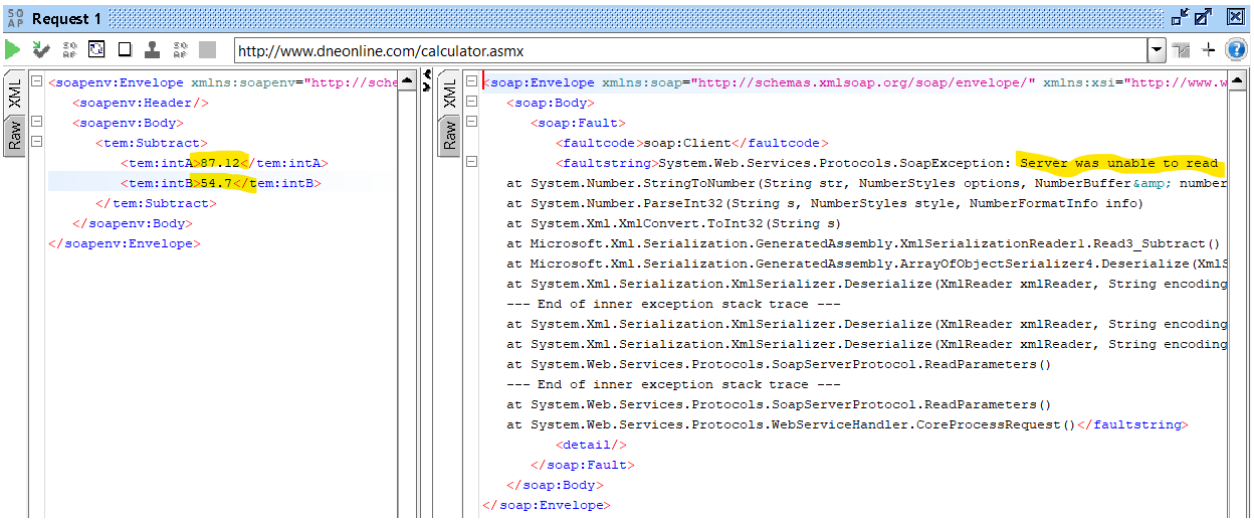
Tegelik tulemus: Ebaõnnestus (test failed)

Järeldus: Süsteem ei toeta väga suuri täisarve.



Test 5 – Subtract (87.12 – 54.7)

Oodatav tulemus: 32.42

Tegelik tulemus: Ebaõnnestus (test failed)

Järeldus: Kümnendmurdudega lahutamine ei ole toetatud.